

A Tour of C++

Personal Notes

Mihir Rao

Contents

1	The Basics	2
1.1	Programs	2
1.2	Functions	4
1.3	Types, Variables, and Arithmetic	5
1.4	Scope and Lifetime	7
1.5	Constants	7
1.6	Pointers, Arrays, and References	8
1.7	Tests	11
1.8	Mapping to Hardware	12
1.9	Advice	14

1 The Basics

This chapter introduces C++’s notation, its model of memory and computation, and the mechanisms that organize code into a program. The facilities described here are those C++ shares with C and support what is often called procedural programming; the abstraction and generic-programming facilities build on top of them.

1.1 Programs

C++ is a compiled language. Source text is processed by a compiler into object files, which a linker combines into an executable program. A program typically spans many source files.

An executable is built for one hardware and operating-system combination and is not portable between them. Portability in C++ refers to source portability: the same source compiles and runs on a variety of systems.

The ISO standard defines two kinds of entities. *Core language features* include the built-in types such as `char` and `int` and the control structures such as `for` and `while`. *Standard-library components* include containers such as `vector` and `map` and I/O operations such as `<<` and `getline`. The library is ordinary C++ code, implemented in C++ apart from trivial uses of machine code for tasks such as thread context switching, which is itself evidence that the language is expressive and efficient enough for demanding systems work.

Definition: C++ is *statically typed*: the type of every object, value, name, and expression must be known to the compiler at its point of use. A type determines both the operations applicable to an object and its layout in memory.

1.1.1 Hello, World!

The minimal program defines a single function and does nothing:

```
1 int main() { }
```

Curly braces express grouping, here delimiting the function body, and `///` begins a comment running to the end of the line. Every program must have exactly one global `main`, and execution starts there. The `int` returned by `main` is the program’s result to the system: zero or no value indicates success, a nonzero value indicates failure. Unix-like environments act on that value; Windows environments rarely do.

Producing output requires the standard library:

```
1 import std;

2 int main()
3 {
4     std::cout << "Hello, World!\n";
5 }
```

The `import std;` directive makes the standard-library declarations available. The operator `<<`, read “put to,” writes its second argument to its first, so the string literal is written to the standard output stream `std::cout`. Within a string literal a backslash followed by another character denotes a single special character, and `'\n'` is a newline. The `std::` qualification says that `cout` is found in the standard-library namespace.

***Note:** Importing the whole standard library as a module is new in C++20 and is not yet universally available. The older form `#include <iostream>` has worked on every implementation since 1998 and remains the reliable fallback.*

Essentially all executable code lives in functions called directly or indirectly from `main`:

```
1 import std;

2 using namespace std;    // make names from std visible without std::

3 double square(double x)
4 {
5     return x * x;
6 }

7 void print_square(double x)
8 {
9     cout << "the square of " << x << " is " << square(x) << "\n";
10 }

11 int main()
12 {
13     print_square(1.234);
14 }
```

A return type of `void` indicates that a function returns no value.

1.2 Functions

Calling a function is the main way of getting something done; defining one specifies how an operation is carried out. A function cannot be called before it has been declared.

A declaration gives the function's name, the type of the value returned if any, and the number and types of its arguments. The return type precedes the name and the argument types follow it in parentheses:

```
1 Elem* next_elem();      // no argument; returns a pointer to Elem
2 void exit(int);         // int argument; returns nothing
3 double sqrt(double);    // double argument; returns a double
```

Argument passing has the semantics of initialization, so argument types are checked and implicit conversions applied where necessary. A call `sqrt(2)` converts its argument to `double`, while `sqrt("three")` is rejected at compile time. The value of this checking is easy to underestimate.

A declaration may name its arguments as documentation for the reader; unless the declaration is also a definition, the compiler ignores those names.

Definition: The *type of a function* is its return type followed by the sequence of its argument types in parentheses. For a member function of a class, the name of the class is also part of the function type.

Comprehensible code is the first step toward maintainable code, and the first step toward comprehensibility is breaking a computation into meaningful chunks and naming them. Functions and classes so named become the vocabulary of a program, just as built-in and user-defined types are the vocabulary of its data. The standard algorithms are a good starting vocabulary, and larger computations compose from there.

The number of errors in code correlates strongly with its volume and complexity, and both are reduced by using more and shorter functions. Making a fragment into a function forces us to name the activity and document its dependencies. Difficulty finding a suitable name is good evidence of a design problem.

1.2.1 Overloading

Several functions may share a name if their argument types differ; the compiler selects the most appropriate one for each call.

```
1 void print(int);
2 void print(double);
3 void print(string);
```

```

4 void user()
5 {
6     print(42);           // calls print(int)
7     print(9.65);        // calls print(double)
8     print("Barcelona"); // calls print(string)
9 }

```

When two candidates are both viable and neither is better than the other, the call is ambiguous and the compiler reports an error. Given `print(int, double)` and `print(double, int)`, the call `print(0, 0)` is rejected on exactly those grounds.

***Note:** Overloading is essential to generic programming, but it carries an obligation: every function of a given name should implement the same semantics. Each `print` above prints its argument, and nothing else.*

1.3 Types, Variables, and Arithmetic

Every name and every expression has a type that determines the operations permitted on it. A declaration is a statement that introduces an entity and specifies its type. Four notions are worth separating precisely.

Definition 1.1 (Type, object, value, variable). A *type* defines a set of possible values together with a set of operations. An *object* is a region of memory holding a value of some type. A *value* is a set of bits interpreted according to a type. A *variable* is a named object.

The fundamental types correspond directly to hardware facilities. Common examples are `bool` with values `true` and `false`; `char`, holding a character; `int`, holding an integer; `double`, a double-precision floating-point number; and `unsigned`, a non-negative integer intended for bitwise operations.

Each has a fixed size that determines its range. A `char` is the natural size to hold a character on a given machine, typically an eight-bit byte, and the sizes of other types are multiples of it. Sizes are implementation-defined and obtained with the `sizeof` operator: `sizeof(char)` is 1 by definition and `sizeof(int)` is commonly 4. When a specific size is required, use a standard-library alias such as `int32_t`.

Numeric literals are floating-point or integer. A floating-point literal is recognized by a decimal point or an exponent. Integer literals are decimal by default; a `0b` prefix gives binary, `0x` gives hexadecimal, and a leading `0` gives octal. A single quote may be used as a digit separator to make long literals readable, as in `3.14159'26535'89793`.

1.3.1 Arithmetic

The arithmetic operators apply to appropriate combinations of the fundamental types: `x + y`, `x - y`, `x * y`, `x / y`, and the integer remainder `x % y`, together with unary `+x` and `-x`. The comparison operators are `x == y`, `x != y`, `x < y`, `x > y`, `x <= y`, and `x >= y`.

The logical operators divide into two groups. The bitwise operators `x & y`, `x | y`, `x ^ y`, and `~x` yield a result of the operand type, applying the operation to each bit. The operators `x && y`, `x || y`, and `!x` simply yield `true` or `false`.

In assignments and arithmetic, C++ performs all meaningful conversions among the basic types so they can be mixed freely. These *usual arithmetic conversions* aim to compute an expression at the highest precision of its operands, so adding a `double` to an `int` uses double-precision arithmetic.

Example: Given `double d = 2.2;` and `int i = 7;`, the assignment `d = d + i;` promotes `i` and stores 9.2. The reverse, `i = d * i;`, computes in double precision and then truncates to an `int`, silently discarding the fractional part.

Beyond the conventional operators, C++ offers compound forms for modifying a variable: `x += y`, `x -= y`, `x *= y`, `x /= y`, `x %= y`, and the increment and decrement operators `++x` and `--x`. Note that `=` assigns while `==` tests equality.

***Note:** Evaluation order is left to right for `x.y`, `x->y`, `x(y)`, `x[y]`, `x << y`, `x >> y`, `x && y`, and `x || y`, and right to left for assignments. For historical reasons connected to optimization, the order of evaluation of other expressions, and of function arguments, is unspecified. Avoid expressions whose meaning depends on it.*

1.3.2 Initialization

An object must be given a value before it is used. C++ offers both the traditional `=` notation inherited from C and a universal form based on curly-brace-delimited initializer lists:

```
1 double d1 = 2.3;
2 double d2 {2.3};
3 double d3 = {2.3};           // the = is optional with { ... }

4 complex<double> z2 {d1, d2};
5 vector<int> v {1, 2, 3, 4, 5, 6};
```

When in doubt prefer the `{}` form, if only because it rejects conversions that lose information. A *narrowing conversion* such as `double` to `int` is permitted and applied implicitly with `=`, but not with braces: `int i1 = 7.8;` quietly yields 7, whereas `int i2 {7.8};` is an error. Implicit narrowing is

part of the price paid for C compatibility.

A constant cannot be left uninitialized, and a variable should be left uninitialized only in rare circumstances. The practical rule is not to introduce a name until there is a suitable value for it.

Where the type can be deduced from the initializer, it need not be stated:

```
1 auto b = true;      // a bool
2 auto ch = 'x';      // a char
3 auto i = 123;       // an int
4 auto d = 1.2;       // a double
5 auto z = sqrt(y);   // whatever sqrt(y) returns
```

With **auto** the = form is usual, since no troublesome conversion is involved. Reasons to name a type explicitly include a large scope where the type should be visible to readers, an initializer whose type is not obvious, and a wish to be explicit about range or precision. Otherwise **auto** avoids redundancy and long type names, which matters most in generic code where exact types can be hard to know and tedious to write.

1.4 Scope and Lifetime

A declaration introduces its name into a scope. *Local scope* belongs to a name declared in a function or lambda, and extends from the point of declaration to the end of the enclosing block delimited by braces; function argument names are local. *Class scope* belongs to a member name defined in a class outside any function, lambda, or enumeration, and extends from the opening brace of the enclosing declaration to the matching one. *Namespace scope* belongs to a name defined in a namespace outside any function, lambda, class, or enumeration, and extends to the end of the namespace. A name not declared inside any other construct is *global* and belongs to the global namespace.

Objects need not be named. Temporaries and objects created with **new** have no name of their own.

An object is constructed before use and destroyed at the end of its scope. A namespace object is destroyed at the end of the program. A member is destroyed when the object containing it is destroyed. An object created by **new** lives until it is destroyed by **delete**.

1.5 Constants

C++ supports two notions of immutability, and the distinction between them is about *when* a value is fixed rather than merely *that* it is fixed.

Definition 1.2 (const and constexpr). **const** means roughly “I promise not to change this value.” The compiler enforces the promise. The value may be computed at run time. **constexpr** means

roughly “to be evaluated at compile time.” The value must be computed by the compiler.

The primary use of `const` is in interfaces, allowing data to be passed through pointers and references without fear of modification. The primary uses of `constexpr` are naming constants, permitting placement of data in read-only memory, and performance.

```
1 constexpr int dmv = 17;           // a named constant
2 int var = 17;                     // not a constant
3 const double sqv = sqrt(var);     // named, possibly computed at run time

4 vector<double> v {1.2, 3.4, 4.5};
5 const double s1 = sum(v);         // OK: evaluated at run time
6 constexpr double s2 = sum(v);    // error: sum(v) is not a constant expression
```

For a function to be usable in a constant expression it must be declared `constexpr` or `constexpr`. A `constexpr` function may also be called with non-constant arguments, in which case its result is simply not a constant expression; this saves us from writing essentially the same function twice. When a function should *only* ever be evaluated at compile time, declare it `constexpr`.

```
1 constexpr double square(double x) { return x * x; }

2 constexpr double max1 = 1.4 * square(17); // OK: a constant expression
3 constexpr double max2 = 1.4 * square(var); // error: var is not a constant
4 const double max3 = 1.4 * square(var);    // OK: evaluated at run time
```

Functions declared `constexpr` or `constexpr` are C++’s version of pure functions: they have no side effects and use only the information passed as arguments. In particular they cannot modify non-local variables, though they may have loops and local variables of their own.

Constant expressions are required by language rules in a few places, including array bounds, case labels, and template value arguments. Elsewhere, compile-time evaluation matters for performance. Independent of performance, immutability is an important design concern in its own right.

1.6 Pointers, Arrays, and References

The most fundamental collection of data is a contiguously allocated sequence of elements of the same type, called an *array*. This is essentially what the hardware offers.

```
1 char v[6];           // array of 6 characters
```

```
2 char* p;           // pointer to character

3 char* q = &v[3];   // q points to v's fourth element
4 char x = *q;       // *q is the object that q points to
```

In a declaration `[]` means “array of” and `*` means “pointer to.” All arrays have 0 as their lower bound, so `v` has six elements `v[0]` through `v[5]`, and the size of an array must be a constant expression. In an expression, prefix `*` means “contents of” and prefix `&` means “address of.”

Traversal by explicit index is available, but for the common case of visiting every element in order the range-**for** statement says what is meant with less room for error:

```
1 void print()
2 {
3     int v[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

4     for (auto i = 0; i != 10; ++i) // explicit loop variable
5         cout << v[i] << '\n';

6     for (auto x : v)                // for each element of v
7         cout << x << '\n';

8     for (auto x : {10, 21, 32, 43}) // for each integer in the list
9         cout << x << '\n';
10 }
```

The range-**for** loop reads as “for every element of `v`, from first to last, place a copy in `x`.” No array bound need be given when an array is initialized from a list. To modify the elements rather than copy them, declare the loop variable a reference:

```
1 void increment()
2 {
3     int v[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

4     for (auto& x : v) // add 1 to each element of v
5         ++x;
6 }
```

Definition: In a declaration the unary suffix `&` means “reference to.” A reference resembles a pointer, except that no prefix `*` is needed to access the value referred to, and a reference cannot be made to refer to a different object after initialization.

References are particularly useful for function arguments. Declaring `void sort(vector<double>& v);` guarantees that a call `sort(my_vec)` sorts `my_vec` itself rather than a copy of it. When an argument should not be modified but copying it would be costly, pass a reference to `const`, as in `double sum(const vector<double>&);`. Functions taking `const` references are very common.

Operators such as `&`, `*`, and `[]` are called *declarator operators* when used in declarations: `T a[n]` declares an array of n objects of type `T`, `T* p` a pointer to `T`, `T& r` a reference to `T`, and `T f(A)` a function taking an `A` and returning a `T`.

1.6.1 The Null Pointer

We try to ensure that a pointer always points to an object, so that dereferencing it is valid. When there is no object to point to, or when we need to represent “no object available,” we give the pointer the value `nullptr`. There is a single `nullptr` shared by all pointer types, and it is a pointer rather than an integer: assigning it to an `int` is an error.

It is often wise to check that a pointer argument actually points to something:

```
1 // count the occurrences of x in p[], which is assumed to be a
2 // zero-terminated array of char, or nothing at all
3 int count_x(const char* p, char x)
4 {
5     if (p == nullptr)
6         return 0;
7
8     int count = 0;
9     while (*p) {
10         if (*p == x)
11             ++count;
12         ++p;
13     }
14     return count;
15 }
```

Incrementing a pointer advances it to the next element of an array. The argument is declared `const char*` because the characters of a string literal are immutable, which is what allows a call such as `count_x("Hello!", 'l')`.

Note: A test of a numeric value is equivalent to comparing it with 0, and a test of a pointer is equivalent to comparing it with `nullptr`; so `while (*p)` and `while (*p != 0)` mean the same thing. In older code `0` or `NULL` appears instead of `nullptr`; prefer `nullptr`, which eliminates the confusion between integers and pointers. There is no null reference: a reference must refer to a valid object.

1.7 Tests

C++ provides a conventional set of selection and looping statements. A function that prompts for a yes-or-no answer illustrates the basic shape, and the `>>` operator, read “get from,” which reads from the standard input stream `cin`:

```
1 bool accept()
2 {
3     cout << "Do you want to proceed (y or n)?\n";

4     char answer = 0;    // a value that will not appear on input
5     cin >> answer;

6     if (answer == 'y')
7         return true;
8     return false;
9 }
```

The type of the right-hand operand of `>>` determines what input is accepted, and that operand is the target of the operation. Note that `answer` is defined where it is needed and not before: a declaration may appear anywhere a statement may.

Testing a value against a set of constants is the job of the `switch` statement. Its case labels must be distinct constants, and if the value matches none of them the `default` is chosen; if there is no `default`, no action is taken.

```
1 bool accept2()
2 {
3     cout << "Do you want to proceed (y or n)?\n";

4     char answer = 0;
5     cin >> answer;

6     switch (answer) {
7     case 'y':
8         return true;
9     case 'n':
10        return false;
11    default:
12        cout << "I'll take that for a no.\n";
13        return false;
14 }
```

15 }

A case need not be left by returning from the enclosing function. To continue execution with the statement following the **switch**, use a **break** statement. Several case labels may share a single action, which is how a parser accepts both 'u' and 'n' for the same movement.

An **if** statement, like a **for** statement, can introduce a variable and test it:

```
1 void do_something(vector<int>& v)
2 {
3     if (auto n = v.size(); n != 0) {
4         // ... reached when n != 0 ...
5     }
6 }
```

Here *n* is defined for use within the statement, initialized from *v.size()*, and tested immediately after the semicolon. A name declared in a condition is in scope on both branches. The purpose, as with the **for** statement, is to limit the scope of the variable, which improves readability and reduces errors.

The most common case is a test against 0 or **nullptr**, and there the condition may be left implicit; write **if (auto n = v.size())** and prefer this terser form where it applies.

1.8 Mapping to Hardware

C++ offers a direct mapping to hardware. Each fundamental operation is implemented by what the hardware offers, typically a single machine instruction; adding two **ints** executes an integer add. An implementation sees memory as a sequence of locations into which it can place typed objects and address them with pointers, and a pointer is represented in memory as a machine address. That an array looks much like this picture is no accident: an array is C++'s basic abstraction of a contiguous sequence of objects in memory.

***Note:** This simple mapping of language constructs onto hardware is the source of the low-level performance for which C and C++ have long been known. The basic machine model of both languages is grounded in computer hardware rather than in some form of mathematics.*

1.8.1 Assignment

Assignment of a built-in type is a simple machine copy. After **int x = 2;**, **int y = 3;**, and **x = y;**, we have **x == y**, and the two objects remain independent: changing *y* afterwards does not affect *x*.

Unlike Java or C#, but like C, this holds for all types, not only for `int`.

Sharing a value must be requested explicitly, and pointers and references request it in different ways. Assigning one pointer to another copies the address, so afterwards both point to the same object:

```
1 int x = 2;
2 int y = 3;
3 int* p = &x;
4 int* q = &y;
5 p = q;           // p becomes &y; now p == q and *p == *q
```

A reference is also represented as a machine address, but the language rules differ: assignment to a reference does not change what it refers to, it assigns to the referenced object.

```
1 int x = 2;
2 int y = 3;
3 int& r = x;    // r refers to x
4 int& r2 = y;   // r2 refers to y
5 r = r2;       // read through r2, write through r: x becomes 3
```

Accessing the value behind a pointer requires `*`; for a reference it is implicit. After `x = y` we have `x == y` for every built-in type and every well-designed user-defined type offering assignment and equality.

1.8.2 Initialization

Initialization differs from assignment. For an assignment to work correctly the assigned-to object must already hold a value; the task of initialization is to turn an uninitialized piece of memory into a valid object. For almost all types, reading from or writing to an uninitialized variable is undefined.

Example: Consider references. Writing `int& r {x};` binds `r` to `x`, after which `r = 7;` assigns through the reference. An uninitialized `int& r2;` is an error, and fortunately so: were it allowed, `r2 = 99;` would write to an unspecified memory location.

The `=` notation may also be used to initialize a reference, as in `int& r = x;`. Despite appearances this is still initialization, binding `r` to `x` rather than performing any value copy.

The distinction matters greatly for user-defined types such as `string` and `vector`, where an assigned-to object already owns a resource that must eventually be released. Argument passing and function return have the semantics of initialization, which is how pass-by-reference is obtained.

1.9 Advice

The chapter closes with guidance drawn from the C++ Core Guidelines. The recurring themes are worth stating compactly. Prefer the standard library to the built-in features used directly, and reach for libraries before writing fundamental machinery by hand. Package meaningful operations as carefully named functions, keep each function to a single logical operation, and keep functions short. Declare a function `constexpr` if it may need compile-time evaluation and `constexpr` if it must have it.

For data, prefer immutability, avoid magic constants in favor of symbolic ones, and avoid narrowing conversions. Keep scopes small and declare a variable only once there is a value to initialize it with. Prefer the brace-initializer syntax for declarations with a named type, use `auto` to avoid repeating type names, and reserve `unsigned` for bit manipulation rather than as a claim that a value is non-negative. Keep pointer use simple, and write `nullptr` rather than `0` or `NULL`. Prefer range-`for` loops to loops with an explicit loop variable.

***Note:** Two closing points of temperament. Do not panic: not every detail of C++ needs to be known to write good programs, and the language rewards a focus on programming techniques over language features. And do not say in comments what the code already states clearly; comments should state intent.*

Source. These notes synthesize Chapter 1, “The Basics,” of Bjarne Stroustrup’s *A Tour of C++*, Third Edition (Addison-Wesley, 2022).